### *HOW TO AVOID HARDCODING ISSUERS OR CREDENTIAL FORMATS IN POLICIES*

Policy authors are sometimes tempted to embed issuer identifiers, credential schemas, or claim names directly in policies. This creates fragile authorization logic that depends heavily on specific external organizations or credential formats. ACME prevents this by using its trust store and signal platform to keep policy logic separate from the details of credential verification.

As noted, ACME depends on the Policy Information Point (PIP) introduced in earlier chapters to do this effectively. The PIP integrates with ACME's trust store, which records the issuers ACME trusts and the types of credentials they issue. The PIP provides this information to PEPs and PDPs so that policies don't need to include issuer knowledge. The PEP uses the trust store data when verifying presentations, checking signatures, and confirming that the credential was issued by an authorized issuer. After verifying and normalizing the claims, the PEP sends only the resulting attributes to the PDP. The PDP evaluates policies based on these attributes and information in the PIP without needing to know anything about credentials issuers, formats, or revocation details.

To keep policy logic clean and stable:

1. **Keep issuer trust in the trust store, not in policy.** Policies should reference information managed by the PIP, such as `context.trusted_certification_bodies`. The PIP maintains this information; PDPs only consume it.
2. **Normalize all credentials before policy evaluation.** The PEP and PIP convert raw VC claim names into stable internal attributes. Policies reference attributes such as `principal.certification` rather than schema-dependent fields.
3. **Express semantics, not formats.** A policy should require an accredited auditing firm rather than encoding a specific issuer name or credential structure. Semantics remain stable even when schemas evolve.
4. Use indirect references for claim sources. If different credential types can provide the same attribute, map them into a single normalized